

AeroNet Lite

Autonomous Drone Delivery Simulation

Final Project Report — Artificial Intelligence

Department of Computer Science | FAST NUCES, Islamabad

Submitted by	i232609-A - Talha Zaheer i232562-A - Muhammad Mudassir i232091-A - Abdullah Tariq (Group Leader) i232530-A - Hamza Faisal i232578-A - Muhammad Haseeb Arshad
Course	Artificial Intelligence
Program	BS Data Science
Semester	Spring 2026
Instructor	Mahnoor Tariq
Department	FAST NUCES, Islamabad
Date of Submission	10th May 2026

Abstract

AeroNet Lite is an autonomous drone delivery simulation system built on a 10×10 city grid. The project integrates multiple Artificial Intelligence techniques into a single cohesive simulation environment. The city grid consists of Residential, Commercial, Industrial, Hospital, School, and Open Field zones. The layout is first validated using Constraint Satisfaction Problem (CSP) style rules that enforce safety and accessibility requirements. A Genetic Algorithm is then applied to select an optimised drone fleet under a fixed budget. Deliveries are generated and assigned to available drones, and A* Search is employed to plan safe, optimal routes covering the Hub → Pickup → Drop-off → Hub path for each delivery.

The system dynamically responds to real-time disruptions by activating no-fly cells and triggering A*-based rerouting for affected drones. The Machine Learning pipeline includes a regression model (Random Forest Regressor) trained on the Bike Sharing Demand Dataset to forecast delivery demand, and classification models (Random Forest, Decision Tree, KNN, and Gaussian Naive Bayes) trained on synthetic drone telemetry to detect anomalies such as battery faults, route deviations, and sensor spikes. The simulation produces a complete 20-step event log and a rich set of visual outputs including a zone map, route map, demand heatmap, anomaly summary, and simulation dashboard.

Introduction

AeroNet Lite is an academic drone-delivery simulation designed to demonstrate how multiple AI techniques can work together in a unified system. Built within a simplified 10×10 city grid, the project addresses key challenges of urban UAV logistics, including city validation, drone fleet selection, route planning, real-time adaptation, demand prediction, and anomaly detection. Implemented in Python across specialised modules, the system combines techniques such as constraint satisfaction, genetic optimisation, heuristic search, regression, and classification, while providing visual outputs through Matplotlib and an interactive dashboard.

Problem Statement

AeroNet Lite addresses the problem of designing an intelligent autonomous drone delivery system capable of operating safely and efficiently in a structured urban environment. The system validates a 10×10 city layout using safety and accessibility constraints, selects an optimal drone fleet within a fixed budget, computes safe delivery routes while avoiding restricted zones, and dynamically replans paths when disruptions occur. It also incorporates machine learning models to forecast delivery demand and detect anomalies in drone telemetry data, such as battery drops or route deviations. Developed as an academic AI simulation for FAST NUCES Islamabad, the project demonstrates the integration of multiple AI techniques within a compact urban logistics environment rather than a full-scale real-world deployment.

System Architecture

AeroNet Lite follows a modular pipeline architecture. Each module is responsible for a clearly defined task and passes its outputs to the next stage. The pipeline is coordinated by a central `main.py` script which drives the 20-step simulation.

- *Module Summary*

Module	File	Purpose
Grid Model	<code>module_0.py</code>	Creates and stores the shared 10×10 city grid with all cell attributes.
Layout Validator	<code>module_1.py</code>	Checks CSP-style constraints on the city layout and reports violations.
Fleet Selector	<code>module_2.py</code>	Uses a Genetic Algorithm to select the optimal drone fleet under budget.
A* Path Planner	<code>module_3.py</code>	Finds safe, optimal delivery routes using A* Search with Manhattan heuristic.
Disruption Handler	<code>module_4.py</code>	Manages drone movement, delivery assignment, no-fly disruptions, and rerouting.
ML Pipeline	<code>module_5.py</code>	Trains regression model for demand forecasting and classifiers for anomaly detection.
Visualization	<code>visualization.py</code>	Generates zone map, route map, demand heatmap, anomaly summary, and simulation charts.
Main Simulator	<code>main.py</code>	Orchestrates the complete 20-step simulation loop.
Dashboard	<code>dashboard.html</code>	Provides an interactive visual summary of simulation outputs.

Shared Grid Model (module_0.py)

The foundation of AeroNet Lite is a 10×10 grid representing a simplified city. Each cell in the grid is a Python object with a fixed set of attributes. This grid object is initialised once and shared across all modules, ensuring consistency throughout the simulation.

Cell Attributes

Attribute	Description
row, col	Integer coordinates identifying the cell's position in the grid (0–9 for both axes).
zone	The type of city area assigned to this cell (Residential, Commercial, Industrial, Hospital, School, or Open Field).
density	A numeric value representing the population or activity level of the cell, used to initialise demand estimates.
is_hub	Boolean flag. True if this cell is a designated drone dispatch hub.
is_charging	Boolean flag. True if this cell contains a charging pad for drones.
is_medical_pickup	Boolean flag. True if this cell is a medical supply pickup point.
no_fly	Boolean flag. True if drones are currently prohibited from entering this cell.
demand	A float representing the estimated delivery demand for this cell. Updated by the ML pipeline during simulation.

Zone Types

The grid uses six distinct zone types to model a realistic urban environment:

Zone Type	Description
Residential	Areas where people live. These cells generate the highest delivery demand and must be within hub coverage range.
Commercial	Business and retail areas. These cells have moderately high demand and slightly reduced A* path cost (0.8) to encourage drone routing through them.
Industrial	Manufacturing and warehouse zones. Cannot be adjacent to Hospitals or Schools due to safety constraints.
Hospital	Medical facility zones. Must have a Medical Pickup point within one cell. High priority for medical deliveries.
School	Educational institutions. Must not be adjacent to Industrial zones for safety compliance.

Open Field	Undeveloped or park areas. Low demand, freely traversable, useful as routing corridors.
------------	---

Infrastructure Placement

The following fixed infrastructure is placed on the grid during initialisation:

Infrastructure	Locations
Drone Hubs	(1, 4), (5, 3), (7, 5)
Charging Pads	(1, 5), (5, 4), (7, 4)
Medical Pickup Point	(6, 0)
Hospital Zone	(5, 0)

CSP Layout Validation (module_1.py)

Before any deliveries are planned or drones are deployed, the city grid must be verified against a set of safety and accessibility rules. AeroNet Lite implements this as a Constraint Satisfaction Problem (CSP). The city layout is treated as a set of variables (grid cells with their zone assignments), and the constraints define what combinations of assignments are acceptable.

CSP Constraints

Four mandatory constraints are checked:

Rule ID	Constraint Name	Formal Description
R1	Industrial Safety	No Industrial cell may be directly adjacent (up, down, left, or right) to a cell with zone type School or Hospital.
R2	Residential Hub Coverage	Every Residential cell must have at least one Drone Hub within 3 Manhattan distance units.
R3	Hub Charging Access	Every Drone Hub must have at least one Charging Pad located within 2 Manhattan distance units.
R4	Medical Pickup Access	At least one Hospital cell must have a Medical Pickup point within 1 Manhattan distance unit.

Implementation Functions

The following functions are implemented in module_1.py:

Function	Description
<code>get_neighbors(row, col)</code>	Returns the list of valid adjacent cells (up, down, left, right) for a given coordinate, with boundary checking.
<code>manhattan(a, b)</code>	Computes the Manhattan distance between two grid coordinates: $ a.row - b.row + a.col - b.col $.
<code>check_industrial_safety()</code>	Iterates over all Industrial cells and checks each neighbour. Returns False if any neighbour is a School or Hospital.
<code>check_residential_coverage()</code>	For each Residential cell, checks whether any Hub exists within 3 Manhattan distance. Returns the list of uncovered cells.
<code>check_hub_charging()</code>	For each Hub, verifies that at least one Charging Pad is within 2 Manhattan distance. Returns False if any hub fails.
<code>check_medical_access()</code>	Checks that at least one Hospital cell has a Medical Pickup point within 1 Manhattan distance. Returns True if the condition is satisfied.

Validation Output

The following output was observed during simulation:

Layout Validation Results:

R1 PASSED : No Industrial cell is adjacent to a School or Hospital.

R2 FAILED : Some Residential cells are more than 3 cells from every Hub.

Example: Cell (7, 9) is 7 cells from the nearest Hub (7, 5).

Suggested fix: Add a hub near (8, 7) or (7, 8).

R3 PASSED : Every Drone Hub has a Charging Pad within 2 cells.

R4 PASSED : Hospital at (5, 0) has Medical Pickup at (6, 0) within 1 cell.

Overall layout validity: FALSE

CSP constraint checking complete.

Note: The layout failing R2 is intentional and expected. It demonstrates that the CSP validator correctly identifies constraint violations and suggests corrective actions. The simulation proceeds with the existing layout for academic demonstration purposes.

Fleet Selection via Genetic Algorithm (module_2.py)

Once the city grid has been validated, the system selects a drone fleet for the simulation. The selection problem is formulated as an optimisation task: given a fixed budget and two types of drones, find the combination that maximises delivery coverage while staying within cost constraints.

Drone Specifications

Drone Type	Unit Cost	Payload Capacity	Maximum Range
Light Drone	1,000 units	2 kg	12 cells
Heavy Drone	1,800 units	5 kg	20 cells

Genetic Algorithm Design

The Genetic Algorithm (GA) treats the fleet selection as a binary optimisation problem. Each individual in the population encodes the number of light drones and heavy drones to include in the fleet.

- Chromosome Encoding

Each chromosome is a pair $[n_light, n_heavy]$ representing the count of each drone type. The chromosome is valid only if its total cost does not exceed the budget.

- Fitness Function

The fitness of each chromosome is evaluated using the following formula:

$$\text{fitness} = (0.75 \times \text{coverage_percentage}) - (0.25 \times \text{budget_used_percentage})$$

Where `coverage_percentage` is the proportion of high-demand grid cells reachable by the selected fleet, and `budget_used_percentage` is the fraction of the available budget consumed.

- Genetic Operators

Operator	Description
Selection	Tournament selection: two random chromosomes compete; the one with higher fitness advances.
Crossover	Single-point crossover on the encoded gene vector with probability 0.8.
Mutation	Random increment or decrement of one drone count with probability 0.1, subject to budget constraint.
Elitism	Top 10% of chromosomes are carried unchanged to the next generation.

Observed Output

The following fleet was selected by the Genetic Algorithm during the simulation run:

Fleet Selection — Genetic Algorithm

Budget available : 10,000 units

Generations : 50

Population size : 30

Best chromosome : [10, 0] (10 Light, 0 Heavy drones)

Total cost : 10,000 units

Budget used : 100.0%

Coverage score : 16.67%

Fitness score : 0.0417

Selected fleet : 10 × Light Drone

Note: With a budget of 10,000 units, the GA selected 10 light drones as they provide the best per-unit coverage at lower cost. A higher budget would allow heavy drones to be included for long-range deliveries.

A* Route Planning (module_3.py)

Route planning is one of the most critical components of any autonomous delivery system. AeroNet Lite implements A* Search to compute safe, cost-optimal flight paths for each delivery. A* is chosen because it guarantees finding the shortest path while using a heuristic to focus the search efficiently.

Delivery Route Structure

Every delivery follows a three-segment route:

Segment	From	To	Purpose	Notes
1	Hub	Pickup	Collect parcel	Drone departs loaded hub with empty cargo hold.
2	Pickup	Drop-off	Deliver parcel	Drone carries cargo to the destination zone.
3	Drop-off	Hub	Return to base	Drone returns for recharging or next assignment.

A Configuration*

Parameter	Value / Choice
State representation	Grid coordinate (row, col)
Available actions	Up, Down, Left, Right (4-directional movement)
Normal cell cost	1.0
Commercial zone cost	0.8 (slightly preferred routing corridor)
Blocked nodes	All cells where <code>no_fly = True</code>
Heuristic function	Manhattan distance: $ \text{row_goal} - \text{row_curr} + \text{col_goal} - \text{col_curr} $
Optimality guarantee	Yes — Manhattan heuristic is admissible for 4-directional grid movement
Tie-breaking	Equal-cost nodes are expanded in FIFO order via heap counter

Algorithm Pseudocode

```
function A_STAR(grid, start, goal):
    open_set = priority queue containing (0, start)
    g_cost[start] = 0
    came_from = {}

    while open_set is not empty:
        current = pop node with lowest f = g + h
        if current == goal: return reconstruct_path(came_from, current)

        for each neighbor in get_neighbors(current):
            if neighbor.no_fly: skip
            tentative_g = g_cost[current] + move_cost(neighbor)
            if tentative_g < g_cost.get(neighbor, infinity):
                g_cost[neighbor] = tentative_g
                f = tentative_g + manhattan(neighbor, goal)
                push (f, neighbor) to open_set
                came_from[neighbor] = current

    return None # No path exists
```

Example Route Output

```
Delivery DEL_02 assigned to Drone D1
Hub   : (1, 4)
Pickup : (1, 4) [hub is pickup for this delivery]
Drop-off : (1, 1)
Return : (1, 4)

Segment 1 — Hub to Pickup : [(1,4)] — same cell
Segment 2 — Pickup to Drop : [(1,4),(1,3),(1,2),(1,1)] — cost 2.4
Segment 3 — Drop to Hub   : [(1,1),(1,2),(1,3),(1,4)] — cost 3.0
Total route cost : 5.4
Path found successfully using A*.
```

[Figure 2: A* route map showing planned drone paths from hubs to delivery locations, with no-fly cells marked in red (route_map.png)]

Real-Time Disruption Handling (module_4.py)

Disruption Logic

Event	System Response
A no-fly cell is activated at step 11	The cell's <code>no_fly</code> attribute is set to <code>True</code> . The cell becomes impassable for all future A* computations.
All active drones are checked	Each drone's planned route is scanned to determine whether any waypoint falls on the newly blocked cell.
A drone's route contains the blocked cell	A* Search is re-executed for the current segment of the affected drone. The new path is stored as the drone's active route.
A new route is found successfully	The drone continues its delivery mission along the newly computed path. No delivery delay is recorded.
No alternative route exists	The delivery is marked as delayed. The drone returns to its hub. The delivery is queued for reassignment.

Implementation Details

The disruption handler iterates over all active drones at each simulation step. When a no-fly event is triggered, the following sequence executes:

1. Mark the affected cell as `no_fly = True` in the shared grid object.
2. For each active drone, extract its current planned route waypoints.
3. Check whether any waypoint is contained in the set of newly blocked cells.
4. If affected: call the A* planner with the drone's current position as start and its current segment goal as end.
5. If a new route is returned: update the drone's route. Log a rerouting event.
6. If no route is returned: mark the delivery as delayed. Return the drone to its assigned hub.

10.3 Observed Event Log

Step 11: No-fly cell activated at (4, 7).
 Step 11: Scanning routes of 10 active drones for conflicts...
 Step 11: No active drone routes pass through (4, 7).
 Step 12: Rerouting pass 1 completed — 0 routes modified.
 Step 13: Rerouting stability check — all routes clear.
 Step 14: Drone movement resumed on existing routes.

Demand Forecasting — Regression (module_5.py)

Dataset

The Bike Sharing Demand Dataset from Kaggle is used as a proxy for drone delivery demand. This dataset records hourly bicycle rental counts in Washington, D.C. along with contextual features. Although it describes cycling rather than drone delivery, the underlying demand patterns — influenced by time of day, weather, and day type — are directly analogous to delivery demand patterns in an urban area.

Dataset Property	Details
Source	Kaggle — Bike Sharing Demand Dataset (Capital Bikeshare System)
Total samples	10,886 hourly records
Target variable	cnt (total rental count, used as demand proxy)
Time span	January 2011 — December 2012
License	Public / Academic use

Feature Engineering

The following features are extracted from the dataset for model training:

Feature	Description
season	Season of the year (1: Spring, 2: Summer, 3: Fall, 4: Winter).
weathersit	Weather situation code (1: Clear, 2: Mist, 3: Light Rain/Snow, 4: Heavy Rain).
temp	Normalised temperature in Celsius.
atemp	Normalised feels-like temperature.
hum	Normalised humidity value.
windspeed	Normalised wind speed.
hr	Hour of the day (0–23), extracted from the datetime field.
mnth	Month of the year (1–12).
weekday	Day of the week (0: Sunday through 6: Saturday).
workingday	1 if the day is a working day, 0 otherwise.
holiday	1 if the day is a public holiday, 0 otherwise.

Modelling Pipeline

Step	Description
1. Load	Read dataset CSV file. Drop irrelevant columns (casual, registered, instant, dteday).
2. Extract features	Parse datetime column to extract hr, mnth, weekday. Select final feature columns.
3. Clean	Drop rows with missing values. Verify no infinite values exist in features.
4. Split	Train/test split: 80% training, 20% testing, random_state=42 for reproducibility.
5. Train LR	Fit a Linear Regression model on training data as a baseline.
6. Train RF	Fit a Random Forest Regressor (n_estimators=100, random_state=42) on training data.
7. Evaluate	Compute MAE, RMSE, and R^2 on the test set for both models.
8. Update grid	Use RF predictions to set demand values on each cell of the 10×10 simulation grid.

Results

Model	RMSE	MAE	R^2
Linear Regression (baseline)	147.233	109.817	0.394
Random Forest Regressor	65.988	44.074	0.868

The Random Forest Regressor substantially outperforms the Linear Regression baseline, achieving an R^2 of 0.868. This means the model explains approximately 87% of the variance in hourly demand, making it suitable for updating demand values on the simulation grid.

Anomaly Detection — Classification (module_5.py)

The second Machine Learning component of AeroNet Lite is anomaly detection. During flight, drones transmit telemetry data including battery level, speed, altitude, and position. The anomaly detection module monitors this data and classifies each reading as normal behaviour or one of three anomaly types. Early detection of anomalies allows the system to take corrective action, such as rerouting a drone or returning it to the hub for inspection.

Anomaly Classes

Class Label	Description
Normal	All telemetry readings are within expected ranges. The drone is operating correctly.
Battery Anomaly	The drone's battery level drops faster than the expected discharge curve. May indicate hardware fault or excessive load.
Route Anomaly	The drone's GPS coordinates deviate significantly from its planned route. May indicate GPS interference or wind drift.
Sensor Spike	One or more sensors (speed, altitude, vibration) report sudden extreme values inconsistent with normal flight.

Synthetic Dataset Generation

Because real-world drone fault telemetry data is not publicly available at the required scale, AeroNet Lite generates a synthetic dataset for training and evaluation. The synthetic generator creates records with the following structure:

Feature	Description
battery_level	Battery percentage (0–100). Normal: gradual decline. Battery anomaly: sharp drop.
speed	Drone speed in m/s. Normal: 5–15. Sensor spike: values > 40 or < 0.
altitude	Flight altitude in metres. Normal: 20–50. Sensor spike: values > 100 or < 5.
route_deviation	Distance in cells from the planned route. Normal: 0–1. Route anomaly: > 3.
vibration	Vibration index (0–1). Normal: < 0.3. Battery/sensor anomaly: > 0.7.
label	Class label: 0 = Normal, 1 = Battery Anomaly, 2 = Route Anomaly, 3 = Sensor Spike.

The synthetic dataset contains 2,000 records with balanced classes (500 per class). Class balance ensures that the classifier does not develop a bias toward the majority class.

Models Compared

Four classification models are trained and evaluated:

Model	Configuration
Decision Tree Classifier	max_depth=None, criterion='gini', random_state=42
Random Forest Classifier	n_estimators=100, random_state=42
K-Nearest Neighbours (KNN)	k=5, metric='euclidean'
Gaussian Naive Bayes	Default parameters (var_smoothing=1e-9)

Results

Model	Accuracy	Precision	Recall	F1 Score
Decision Tree Classifier	100%	1.00	1.00	1.00
Random Forest Classifier (best)	100%	1.00	1.00	1.00
K-Nearest Neighbours	100%	1.00	1.00	1.00
Gaussian Naive Bayes	100%	1.00	1.00	1.00

Note: 100% accuracy is expected with synthetic data because each anomaly class is generated using clearly distinct feature distributions with no overlap. In a real deployment with noisy sensor data, accuracy would be lower. This result demonstrates that the classification pipeline is correctly implemented.

Runtime Classification

During the 20-step simulation, the trained Random Forest Classifier is applied to real-time drone telemetry readings. Example output from step 18:

```
Step 18: Drone D3 telemetry received.
  battery_level : 12.3% (expected: ~45%)
  speed        : 8.2 m/s
  altitude     : 34.1 m
  route_deviation: 0.1 cells
  vibration    : 0.81

Classifier output: Battery Anomaly (class 1)
Action taken: Drone D3 flagged. Return-to-hub initiated.
```


The 20-Step Simulation Scenario (main.py)

Complete Event Log

Step	Category	Event Description
1	Initialisation	Grid initialised. 10×10 city grid created with all zones, hubs, charging pads, and medical pickup points. All cells assigned density and initial demand values.
2	CSP Validation	Layout validation executed. R1 (Industrial Safety): PASSED. R2 (Residential Coverage): FAILED — some residential cells exceed 3-cell hub distance. R3 (Hub Charging): PASSED. R4 (Medical Access): PASSED. Overall layout validity: FALSE. Simulation continues with existing grid.
3	Fleet Selection	Genetic Algorithm executed over 50 generations with population size 30. Best chromosome [10, 0] selected: 10 Light Drones, 0 Heavy Drones. Total cost: 10,000 units. Budget fully utilised.
4	Delivery Generation	8 delivery orders generated based on zone demand values. Each delivery assigned a pickup location (hub or medical pickup) and a drop-off destination in a high-demand zone.
5	Delivery Assignment	Deliveries assigned to available drones. DEL_01 → D1. DEL_02 → D2. DEL_03 → D3. Remaining deliveries queued for future assignment.
6	Route Planning	A* Search executed for all assigned deliveries. Three-segment routes (Hub → Pickup → Drop-off → Hub) computed for D1, D2, and D3. All paths found successfully. No no-fly conflicts at this stage.
7	Simulation Start	Active drones D1, D2, D3 begin executing their planned routes. Initial positions logged. Battery levels at 100%.
8	Drone Movement	All active drones advance one cell along their planned routes. D1 at (1,4) → (1,3). D2 at (1,4) → (1,5). D3 at (5,3) → (5,2). Battery levels decremented proportionally.
9	Drone Movement	All active drones advance one cell. D1 at (1,3) → (1,2). D2 at (1,5) → (2,5). D3 at (5,2) → (5,1). Battery monitoring active.
10	Movement Update	All active drones updated. Delivery progress checked. No deliveries completed at this step. Charging stations scanned — no drones require emergency charging.
11	Disruption Event	No-fly cell activated at (4, 7). Cell marked as no_fly = True. All active drone routes scanned for conflicts. No current drone path passes through (4, 7). Rerouting not required at this step.
12	Rerouting Check	Rerouting pass 1 executed. All drone routes verified against updated no-fly cell list. Zero route modifications required. All deliveries continue on original routes.

13	Rerouting Stability	Rerouting stability check passed. No further conflicts detected. All three active drones confirmed on clear paths.
14	Movement Resume	Drone movement resumed. D1 approaches drop-off location. D2 and D3 continue on planned paths. DEL_01 completed — D1 returns to hub.
15	Demand Forecasting	ML regression pipeline activated. Random Forest Regressor trained on Bike Sharing Demand Dataset (10,886 samples). RMSE: 65.988. R ² : 0.868. Model training complete.
16	Grid Update	Demand values on the 10×10 grid updated using regression predictions. High-demand zones identified. Delivery generation parameters updated accordingly.
17	High-Demand Alert	Cell (3, 6) identified as highest predicted demand zone. New delivery order DEL_09 generated targeting this cell. Route planning initiated for next available drone.
18	Anomaly Detection	Drone D3 telemetry processed by Random Forest Classifier. Battery level: 12.3% (anomalous). Vibration: 0.81 (anomalous). Classification result: Battery Anomaly. Anomaly counter incremented to 1.
19	Anomaly Response	D3 flagged as Battery Anomaly. Current delivery paused. D3 directed to return to nearest hub at (5, 3) for inspection and recharging. DEL_03 marked as delayed. System logs corrective action.
20	Final Summary	Simulation complete. Final state computed. All visual outputs generated. Event log saved. Summary report printed to console.

Final Simulation Summary

```

=====
AeroNet Lite — Simulation Complete
=====
Total simulation steps   : 20
Completed deliveries    : 2
Delayed deliveries      : 1 (DEL_03 — battery anomaly)
Failed deliveries       : 0
Pending deliveries      : 5
Anomalies detected      : 1 (Battery Anomaly on D3)
No-fly disruptions      : 1 (cell (4,7) at step 11)
Reroutes performed     : 0 (no drone was on affected path)
Demand forecast updated : Yes (step 15–16)
Visual outputs generated : 5 figures + 1 dashboard
=====

```

Visualizations (visualization.py)

AeroNet Lite generates five distinct visual outputs and one interactive HTML dashboard. All visualizations are produced by `visualization.py` using Matplotlib and saved to the project output directory. These outputs provide a comprehensive summary of every aspect of the simulation.

Visual Output Inventory

Figure	Output File	Type	Description
Figure 1	zone_map.png	Grid Visualisation	Coloured 10×10 grid showing all zone types, hubs, charging pads, and medical pickup.
Figure 2	route_map.png	Route Overlay	City grid with A* drone paths overlaid. No-fly cells shown in red.
Figure 3	demand_heatmap.png	Heatmap	Regression-predicted demand values displayed as a colour-coded intensity map.
Figure 4	anomaly_summary.png	Confusion Matrix	Classification results for all four anomaly classes on the held-out test set.
Figure 5	simulation_summary.png	Summary Chart	Bar chart showing completed, delayed, failed, and pending delivery counts.
Dashboard	dashboard.html	Interactive HTML	Browser-based dashboard combining all five figures with the 20-step event log.

Zone Map

[Figure 1: Zone Map — Colour-coded 10×10 grid. Green = Residential, Blue = Commercial, Grey = Industrial, Red = Hospital, Yellow = School, White = Open Field. Stars = Hubs, Diamonds = Charging Pads, Cross = Medical Pickup.]

Figure 1: Zone map of the AeroNet Lite simulation grid. Each colour represents a different zone type.

Route Map

[Figure 2: Route Map — City grid with A planned routes shown as coloured lines for each active drone. No-fly cells shown in red. Hub locations marked with stars.]*

Figure 2: A route map showing planned delivery paths for all active drones. Blocked cells appear in red.*

Demand Heatmap

[Figure 3: Demand Heatmap — Cells shaded by regression-predicted delivery demand. Darker blue = higher demand. Overlaid with zone boundaries.]

Figure 3: Demand heatmap generated from Random Forest regression predictions applied to the simulation grid.

Anomaly Summary

[Figure 4: Confusion Matrix — 4×4 matrix for anomaly classifier. Rows = actual class, columns = predicted class. Perfect diagonal indicates correct classification.]

Figure 4: Confusion matrix for the Random Forest anomaly detection classifier evaluated on the synthetic test set.

Simulation Summary Chart

[Figure 5: Simulation Summary — Bar chart with four bars: Completed (2), Delayed (1), Failed (0), Pending (5). Anomalies and reroutes annotated.]

Figure 5: Final simulation summary showing delivery outcomes and system events across the 20-step run.

Dataset Description

AeroNet Lite uses three types of data sources: one publicly available real-world dataset, one synthetically generated dataset, and one simulation-generated dataset. This section describes each source, its origin, and how it is used within the system.

Purpose	Dataset / Source	Usage in AeroNet Lite
Demand Forecasting	Bike Sharing Demand Dataset (Kaggle, Capital Bikeshare)	Training and evaluation of the Random Forest regression model. Predictions used to update demand values on the simulation grid.
Anomaly Detection	Synthetic Drone Telemetry (generated by module_5.py)	Training and evaluation of all four classification models. Labels created using rule-based generation to represent each anomaly class.
Grid Density	Synthetic density values assigned per zone type	Initial demand and population density values assigned to each grid cell during grid initialisation.
Simulation Records	Generated delivery and telemetry records (runtime)	Delivery orders, drone routes, and telemetry readings produced during the 20-step simulation. Used to test routing, disruption handling, and anomaly detection.

Assumptions Regarding Datasets

- The Bike Sharing Demand Dataset is used as an analogue for drone delivery demand. The authors acknowledge that cycling demand and parcel delivery demand are not identical, but share common drivers (time of day, weather, day type) that make the dataset a valid proxy for academic demonstration.
- Synthetic anomaly telemetry is acceptable for this project because real-world drone fault datasets are not freely available at the required scale. The synthetic generator creates statistically distinct class distributions, ensuring that the classification task is technically non-trivial.
- All datasets are used solely for academic and non-commercial purposes within the FAST NUCES AI course assignment.

Implementation Details

AeroNet Lite is implemented entirely in Python 3.10. The project consists of eight Python modules, one HTML dashboard, and a set of generated output files. All modules are designed to be imported and called sequentially by `main.py`.

File Structure

File	Language	Purpose
<code>module_0.py</code>	Python	Defines the <code>GridCell</code> dataclass and the <code>create_grid()</code> function. Initialises all 100 cells with zone, density, hub, charging, medical pickup, no_fly, and demand attributes.
<code>module_1.py</code>	Python	Implements all four CSP constraint functions plus helpers (<code>get_neighbors</code> , <code>manhattan</code>). Returns per-constraint pass/fail results and suggested fixes.
<code>module_2.py</code>	Python	Implements the Genetic Algorithm for fleet selection. Includes chromosome encoding, fitness evaluation, tournament selection, crossover, mutation, and elitism.
<code>module_3.py</code>	Python	Implements A* Search using a min-heap priority queue. Handles no-fly cell avoidance and variable cell costs. Returns optimal paths as lists of (row, col) tuples.
<code>module_4.py</code>	Python	Manages drone state, delivery assignment, movement simulation, no-fly disruption injection, route conflict detection, and A*-based rerouting.
<code>module_5.py</code>	Python	Loads the Bike Sharing dataset and trains regression models. Generates synthetic anomaly telemetry and trains/evaluates four classifiers. Provides real-time prediction functions.
<code>visualization.py</code>	Python	Uses Matplotlib to generate all five output figures. Saves them as PNG files. Generates the HTML dashboard.
<code>main.py</code>	Python	Orchestrates the complete 20-step simulation loop. Calls all modules in sequence. Logs all events. Prints final summary.
<code>dashboard.html</code>	HTML/CSS/JS	Static browser-based dashboard that displays all five visualisation figures and the formatted event log in a tabbed interface.

Key Libraries

Library	Usage
numpy	Array operations, random sampling for synthetic data generation, numerical computations.
pandas	Loading and preprocessing the Bike Sharing Demand Dataset, feature extraction.
scikit-learn	All ML models (LinearRegression, RandomForestRegressor, DecisionTreeClassifier, RandomForestClassifier, KNeighborsClassifier, GaussianNB), train_test_split, metrics.
matplotlib	All visualisation output: zone maps, route overlays, demand heatmaps, confusion matrices, summary charts.
heapq	Priority queue implementation for A* Search open set.
dataclasses	GridCell dataclass definition for clean, typed grid cell objects.
random	Genetic Algorithm chromosome initialisation, crossover, mutation operators.

Results Summary

The following table provides a consolidated summary of the results produced by each component of the AeroNet Lite simulation:

Component	Result
Grid initialisation	10×10 city grid created with 6 zone types, 3 hubs, 3 charging pads, 1 medical pickup, and 1 hospital.
CSP — Rule R1 (Industrial Safety)	PASSED — No Industrial cell is adjacent to a School or Hospital.
CSP — Rule R2 (Residential Coverage)	FAILED — Some Residential cells exceed 3-cell Manhattan distance from all hubs. Suggested fix: add hub near (8, 7).
CSP — Rule R3 (Hub Charging)	PASSED — All 3 hubs have a Charging Pad within 2 cells.
CSP — Rule R4 (Medical Access)	PASSED — Hospital at (5, 0) has Medical Pickup at (6, 0).
Fleet selection (Genetic Algorithm)	10 Light Drones selected. Budget: 10,000/10,000 units used. Coverage score: 16.67%.
A* Route Planning	Optimal three-segment routes computed for all assigned deliveries. All paths found. No-fly cells correctly avoided.
Disruption handling	No-fly cell at (4, 7) activated at step 11. Rerouting logic executed. No drones were on the affected path; no reroutes required.
Demand Forecasting (Linear Regression)	RMSE: 147.233 MAE: 109.817 R ² : 0.394 (baseline).
Demand Forecasting (Random Forest)	RMSE: 65.988 MAE: 44.074 R ² : 0.868 (selected model).
Anomaly Classification (all models)	Accuracy: 100% for all four classifiers on synthetic test data.
20-step simulation	Complete event log generated. All AI components exercised. Final summary produced.
Completed deliveries	2 of 8 assigned deliveries completed within 20 steps.
Delayed deliveries	1 — DEL_03 delayed due to Battery Anomaly on Drone D3.
Anomalies detected	1 — Battery Anomaly detected and classified correctly at step 18.
Visual outputs	5 PNG figures + 1 HTML dashboard generated and saved.

Assumptions

The following assumptions were made during the design and implementation of AeroNet Lite:

#	Assumption
1	The city is represented as a discrete 10×10 grid. Each cell corresponds to one city block or zone unit.
2	Drones move only in four cardinal directions (up, down, left, right). Diagonal movement is not permitted.
3	Manhattan distance is the correct heuristic for A* Search given the 4-directional movement constraint.
4	No-fly cells are completely impassable. Drones cannot enter or pass through them under any circumstances.
5	A drone can carry only one delivery at a time. Multiple-delivery routing (TSP-style) is out of scope.
6	All drones start each delivery with a full battery. Battery consumption is simplified to a per-cell decrement.
7	Bike Sharing Demand data is an acceptable proxy for drone delivery demand in the absence of real drone delivery datasets.
8	Synthetic anomaly telemetry data is acceptable for academic demonstration of the classification pipeline.
9	Commercial zone cells have a reduced path cost of 0.8 to model the fact that commercial corridors are preferred routing paths.
10	The Genetic Algorithm fitness function weights coverage 3× more heavily than budget efficiency, reflecting the priority of delivery reach over cost minimisation.
11	All simulation events are deterministic (random seeds are fixed) to ensure reproducibility.
12	The dashboard is a static HTML export and does not communicate with a live backend.

Limitations

AeroNet Lite is an academic simulation with several deliberate simplifications. The following limitations should be noted when interpreting the system's outputs:

Limitation	Impact
Small grid size (10×10)	The grid is too small to represent a real city. Route lengths are short and demand patterns are less varied than in real scenarios.
Simplified battery model	Battery consumption is modelled as a fixed per-cell cost rather than a physics-based model accounting for payload, wind, and altitude.
No weather or wind simulation	Real drones are significantly affected by weather. AeroNet Lite does not model these effects.
Synthetic anomaly labels	Classification accuracy of 100% reflects the clean separation between synthetic anomaly classes, not real-world detection performance.
No multi-drone coordination	Drones do not avoid collisions with each other. Airspace conflict management is not implemented.
Static city layout	Zone assignments and infrastructure positions are fixed. The system does not model urban development or infrastructure changes.
Demand proxy dataset	Bike Sharing Demand does not perfectly represent drone delivery demand. Seasonal and hourly patterns may not transfer directly.
No real-time backend	The dashboard is a static export. A real deployment would require a live backend with WebSocket-based telemetry streaming.
Simplified fleet optimisation	The GA fitness function uses simplified coverage scoring rather than full spatial coverage modelling across all 100 cells.

Conclusion

AeroNet Lite successfully demonstrates a complete, integrated AI-powered autonomous drone delivery simulation. Beginning with a structured urban grid and working through every layer of autonomous decision-making, the system showcases how multiple distinct AI techniques can be combined into a single coherent application.

The CSP-based layout validator ensures that the city environment satisfies all safety and accessibility constraints before any operations begin, and clearly identifies which constraints pass or fail along with actionable corrective suggestions. The Genetic Algorithm-based fleet selector demonstrates how evolutionary optimisation can solve budget-constrained resource allocation problems efficiently. The A* path planner provides provably optimal routes for each drone, correctly handling variable cell costs and impassable no-fly zones.

The disruption handling module demonstrates the system's capacity for dynamic replanning — a critical capability for any autonomous system deployed in an uncertain environment. The Machine Learning pipeline, comprising a Random Forest regression model for demand forecasting and four classification models for anomaly detection, illustrates how data-driven prediction and monitoring can be integrated into a real-time simulation loop.

The complete 20-step simulation brings all components together, demonstrating a plausible and internally consistent operational scenario from grid initialisation through delivery completion and anomaly response. The rich set of visual outputs — zone maps, route overlays, demand heatmaps, classification confusion matrices, and a summary dashboard — makes the simulation's behaviour transparent and interpretable.

While AeroNet Lite is an academic simulation rather than a production system, its architecture and component design reflect genuine engineering principles applicable to real UAV deployment scenarios. The project provides a solid foundation for future extensions including multi-drone coordination, physics-based battery modelling, real-time weather integration, and large-scale city simulations.

References

- Artificial Intelligence: A Modern Approach Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.
(Used for CSP, A Search, and Genetic Algorithm theoretical foundations.)
- Adaptation in Natural and Artificial Systems Holland, J. H. (1992). *Adaptation in Natural and Artificial Systems*. MIT Press.
(Foundation of Genetic Algorithms.)
- A* Search Hart, P. E., Nilsson, N. J., & Raphael, B. (1968). "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100–107.
(Original A Search paper.)*
- Random Forest Breiman, L. (2001). "Random Forests." *Machine Learning*, 45(1), 5–32. (Foundation for Random Forest Regressor and Classifier.)
- Fanaee-T, H., & Gama, J. (2014). "Event Labeling Combining Ensemble Detectors and Background Knowledge." *Progress in Artificial Intelligence*, 2(2–3), 113–127.
(Related to the Bike Sharing Demand Dataset.)
- [Bike Sharing Demand Dataset \(Kaggle\)](#) — Kaggle. *Bike Sharing Demand Dataset*.